

CSA63 – Threat Intelligence and Network Security

All experiments use Python only (standard library, plus one script using the 'cryptography' package).

Recommended tools: Python 3.x and any editor (IDLE / VS Code / Thonny).

Experiment 1: Caesar Cipher – Understanding Confidentiality

Aim:

To demonstrate the concept of confidentiality (from the CIA triad) by encrypting and decrypting a message using a simple substitution cipher.

Concept:

Confidentiality means only authorized parties can read data. A Caesar cipher shifts each letter by a fixed number of positions, making the original message unreadable to anyone who doesn't know the shift value (the 'key').

Python Code:

```
def caesar_encrypt(text, shift):
    result = ""
    for ch in text:
        if ch.isalpha():
            base = ord('A') if ch.isupper() else ord('a')
            result += chr((ord(ch) - base + shift) % 26 + base)
        else:
            result += ch
    return result

def caesar_decrypt(text, shift):
    return caesar_encrypt(text, -shift)

message = "AttackAtDawn"
shift = 3
enc = caesar_encrypt(message, shift)
dec = caesar_decrypt(enc, shift)

print("Original :", message)
print("Encrypted:", enc)
print("Decrypted:", dec)
```

Sample Output:

```
Original : AttackAtDawn
Encrypted: DwdfnDwGdzq
Decrypted: AttackAtDawn
```

Where to Run:

Any free compiler (Programiz, OnlineGDB, Replit, Trinket, Google Colab). Pure logic, no setup needed.

Experiment 2: File Hashing – Understanding Integrity

Aim:

To demonstrate the concept of integrity by detecting whether a file has been tampered with, using cryptographic hashing.

Concept:

Integrity means data has not been altered without authorization. A hash function (SHA-256) produces a unique fingerprint of a file. If even one character changes, the hash changes completely — this is how systems detect tampering.

Python Code:

```
import hashlib

def file_hash(filename, algo="sha256"):
    h = hashlib.new(algo)
    with open(filename, "rb") as f:
        h.update(f.read())
    return h.hexdigest()

with open("sample.txt", "w") as f:
    f.write("Confidential report v1")

print("Original hash :", file_hash("sample.txt"))

with open("sample.txt", "a") as f:
    f.write(" - tampered!")

print("Modified hash :", file_hash("sample.txt"))
print("Integrity check: FAILED (hash changed) -> file was modified")
```

Sample Output:

```
Original hash : 5a54470e71678d9dbac7ffc51f2cd2a76cc1e8e0d1096bd698eef42229f687d7
Modified hash : 66aa68df2112cb433631ca8675173a88a7e6b2b8a64a4942d01221ee3e4ce2e2
Integrity check: FAILED (hash changed) -> file was modified
```

Where to Run:

Any free compiler. The script creates its own file, so no upload is needed.

Experiment 3: Simple TCP Port Scanner

Aim:

To write a program that checks which network ports are open on a target host.

Concept:

Attackers scan ports to find running services they can exploit (e.g., port 21 = FTP, 80 = HTTP). This experiment recreates that logic in a few lines using Python's socket module, on your own machine only.

Python Code:

```
import socket

def scan_port(host, port):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(0.5)
    result = s.connect_ex((host, port))
    s.close()
    return result == 0

target = "127.0.0.1" # scan your own machine (localhost)
ports = [21, 22, 23, 25, 80, 443, 3306, 8080]

print(f"Scanning {target} ...")
for port in ports:
    status = "OPEN" if scan_port(target, port) else "closed"
    print(f"Port {port}: {status}")
```

Sample Output:

```
Scanning 127.0.0.1 ...
Port 21: closed
Port 22: closed
Port 23: closed
Port 25: closed
Port 80: closed (OPEN if a web server is running locally)
Port 443: closed
Port 3306: closed
Port 8080: closed
```

Where to Run:

Replit or Google Colab only. Needs real socket/network access — Programiz and OnlineGDB block outbound connections and the script will hang or time out.

Experiment 4: Phishing URL Detector

Aim:

To write a program that flags suspicious URLs based on common phishing indicators.

Concept:

Phishing links often disguise their real destination. Simple heuristics — raw IP addresses instead of domain names, '@' symbols, excessive hyphens, or sensitive keywords without HTTPS — can catch many basic phishing attempts.

Python Code:

```
import re

def check_url(url):
    reasons = []
    if re.match(r"https?:\/\/\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}", url):
        reasons.append("Uses raw IP address instead of domain name")
    if "@" in url:
        reasons.append("Contains '@' symbol (can hide real destination)")
    if url.count("-") > 3:
        reasons.append("Too many hyphens in domain (common phishing trick)")
    if any(k in url.lower() for k in ["login", "verify", "update", "secure"]) and "https"
not in url:
        reasons.append("Suspicious keyword without HTTPS")
    return reasons

urls = [
    "https://www.google.com",
    "http://192.168.10.5/login",
    "http://paypal-verify-account.com@evil.com",
]

for u in urls:
    issues = check_url(u)
    verdict = "SUSPICIOUS" if issues else "Looks OK"
    print(f"\nURL: {u}\nVerdict: {verdict}")
    for r in issues:
        print(" -", r)
```

Sample Output:

```
URL: https://www.google.com
Verdict: Looks OK

URL: http://192.168.10.5/login
Verdict: SUSPICIOUS
- Uses raw IP address instead of domain name
- Suspicious keyword without HTTPS
```

URL: `http://paypal-verify-account.com@evil.com`

Verdict: SUSPICIOUS

- Contains '@' symbol (can hide real destination)
- Suspicious keyword without HTTPS

Where to Run:

Any free compiler. Pure string/regex logic, no files or network needed.

Experiment 5: DNS Lookup (OSINT Basics)

Aim:

To resolve domain names to IP addresses — a basic open-source intelligence (OSINT) gathering technique.

Concept:

Before attacking or defending a network, analysts gather public information about it. Resolving a domain to its IP address is one of the simplest OSINT steps, done here with Python's built-in socket module.

Python Code:

```
import socket

domains = ["www.google.com", "www.python.org", "notarealdomain12345.com"]

for d in domains:
    try:
        ip = socket.gethostbyname(d)
        print(f"{d:30s} -> {ip}")
    except socket.gaierror:
        print(f"{d:30s} -> Could not resolve (invalid/unreachable)")
```

Sample Output:

```
www.google.com           -> 142.250.xxx.xxx
www.python.org           -> 151.101.xxx.xxx
notarealdomain12345.com  -> Could not resolve (invalid/unreachable)

(Actual IPs depend on your network; run with internet access.)
```

Where to Run:

Replit or Google Colab only. Needs internet access to resolve domain names — blocked on sandboxed compilers like Programiz/OnlineGDB.

Experiment 6: Indicator of Compromise (IoC) Matching

Aim:

To scan a sample traffic log against a list of known malicious IP addresses (IoCs) and flag matches.

Concept:

Threat intelligence feeds provide IoCs — known bad IPs, domains, or file hashes. Security tools continuously check logs against these lists. This experiment simulates that matching process on two small text files.

Python Code:

```
# Step 1: Create the sample files (so this script runs standalone,
# with no file upload needed - works on any online compiler)
with open("ioc_list.txt", "w") as f:
    f.write("45.33.32.156\n198.51.100.23\n203.0.113.99\n")

with open("sample_traffic.log", "w") as f:
    f.write("2026-07-08 10:01:12 connection from 10.0.0.5\n")
    f.write("2026-07-08 10:02:44 connection from 45.33.32.156\n")
    f.write("2026-07-08 10:03:10 connection from 192.168.1.20\n")
    f.write("2026-07-08 10:04:55 connection from 203.0.113.99\n")

# Step 2: Load IoCs and scan the log
def load_iocs(filename):
    with open(filename) as f:
        return set(line.strip() for line in f if line.strip())

def scan_log(logfile, iocs):
    with open(logfile) as f:
        for line in f:
            for ip in iocs:
                if ip in line:
                    print("ALERT: Known malicious IP found ->", line.strip())

iocs = load_iocs("ioc_list.txt")
print("Loaded", len(iocs), "IoCs")
scan_log("sample_traffic.log", iocs)
```

Sample Output:

```
Loaded 3 IoCs
ALERT: Known malicious IP found -> 2026-07-08 10:02:44 connection from 45.33.32.156
ALERT: Known malicious IP found -> 2026-07-08 10:04:55 connection from 203.0.113.99
```

Where to Run:

Any free compiler. The script creates its sample files (ioc_list.txt, sample_traffic.log) itself, so it runs standalone with no upload required.

Experiment 7: Log File Analyzer

Aim:

To parse a login log file and count failed login attempts per IP address.

Concept:

Real SIEM tools (like Splunk) parse thousands of log lines to spot patterns. This experiment recreates the core idea — reading a text log, extracting fields with regular expressions, and counting events — in plain Python.

Python Code:

```
import re
from collections import Counter

# Create the sample log so the script runs standalone (no upload needed)
with open("login_attempts.log", "w") as f:
    f.write("2026-07-08 09:00:01 LOGIN SUCCESS user=alice ip=10.0.0.5\n")
    f.write("2026-07-08 09:01:15 LOGIN FAILED user=admin ip=203.0.113.99\n")
    f.write("2026-07-08 09:01:20 LOGIN FAILED user=admin ip=203.0.113.99\n")
    f.write("2026-07-08 09:01:25 LOGIN FAILED user=admin ip=203.0.113.99\n")
    f.write("2026-07-08 09:01:30 LOGIN FAILED user=admin ip=203.0.113.99\n")
    f.write("2026-07-08 09:01:35 LOGIN FAILED user=admin ip=203.0.113.99\n")
    f.write("2026-07-08 09:02:00 LOGIN SUCCESS user=bob ip=10.0.0.8\n")

failed_by_ip = Counter()

with open("login_attempts.log") as f:
    for line in f:
        if "LOGIN FAILED" in line:
            match = re.search(r"ip=(\S+)", line)
            if match:
                failed_by_ip[match.group(1)] += 1

print("Failed login attempts by IP:")
for ip, count in failed_by_ip.items():
    print(f" {ip}: {count} attempts")
    if count >= 5:
        print(f" -> ALERT: possible brute-force attack from {ip}")
```

Sample Output:

```
Failed login attempts by IP:
203.0.113.99: 5 attempts
-> ALERT: possible brute-force attack from 203.0.113.99
```

Where to Run:

Any free compiler. The script creates login_attempts.log itself, so it runs standalone with no upload required.

Experiment 8: Dictionary Attack Demo

Aim:

To demonstrate why weak passwords are vulnerable, by cracking a hashed password using a wordlist.

Concept:

A dictionary attack tries common passwords one by one, hashing each guess and comparing it to the stolen hash. This experiment shows, in miniature, why simple passwords like 'letmein' are cracked almost instantly.

Python Code:

```
import hashlib

# Simulate a stolen password hash (a real system would never expose this!)
stolen_hash = hashlib.sha256("letmein".encode()).hexdigest()

wordlist = ["123456", "password", "admin", "letmein", "qwerty"]

print("Attempting dictionary attack on stolen hash...")
for word in wordlist:
    guess_hash = hashlib.sha256(word.encode()).hexdigest()
    if guess_hash == stolen_hash:
        print(f"Password cracked: '{word}'")
        break
else:
    print("Password not found in wordlist.")
```

Sample Output:

```
Attempting dictionary attack on stolen hash...
Password cracked: 'letmein'
```

Where to Run:

Any free compiler. Pure hashlib logic, no files or network needed.

Experiment 9: Firewall Rule Simulator

Aim:

To simulate how a firewall decides whether to allow or block a network packet based on rules.

Concept:

Firewalls check each packet's source IP and port against an ordered rule list, and apply a 'default deny' if nothing matches. This experiment implements that same rule-matching logic in Python, without touching real iptables.

Python Code:

```
rules = [
    {"action": "ALLOW", "ip": "10.0.0.5",      "port": 443},
    {"action": "ALLOW", "ip": "10.0.0.5",      "port": 80},
    {"action": "DENY",   "ip": "203.0.113.99",  "port": None}, # block this IP entirely
    {"action": "DENY",   "ip": None,           "port": 23},  # block telnet from anyone
]

def check_packet(ip, port):
    for rule in rules:
        ip_match = rule["ip"] in (None, ip)
        port_match = rule["port"] in (None, port)
        if ip_match and port_match:
            return rule["action"]
    return "DENY" # default: deny anything not explicitly allowed

packets = [
    ("10.0.0.5", 443),
    ("203.0.113.99", 80),
    ("10.0.0.9", 23),
    ("10.0.0.9", 8080),
]

for ip, port in packets:
    decision = check_packet(ip, port)
    print(f"Packet from {ip}:{port} -> {decision}")
```

Sample Output:

```
Packet from 10.0.0.5:443 -> ALLOW
Packet from 203.0.113.99:80 -> DENY
Packet from 10.0.0.9:23 -> DENY
Packet from 10.0.0.9:8080 -> DENY
```

Where to Run:

Any free compiler. Pure logic, no dependencies.

Experiment 10: Simple Signature-Based Intrusion Detection

Aim:

To write a basic Intrusion Detection System (IDS) that raises an alert when a known attack pattern (repeated failed logins) is seen.

Concept:

Signature-based IDS tools (like Snort) match traffic against known attack patterns. Here, the 'signature' is a simple threshold: 3 or more failed logins from the same IP triggers an alert.

Python Code:

```
from collections import defaultdict

# Create the sample log so the script runs standalone (no upload needed)
with open("login_attempts.log", "w") as f:
    f.write("2026-07-08 09:00:01 LOGIN SUCCESS user=alice ip=10.0.0.5\n")
    f.write("2026-07-08 09:01:15 LOGIN FAILED user=admin ip=203.0.113.99\n")
    f.write("2026-07-08 09:01:20 LOGIN FAILED user=admin ip=203.0.113.99\n")
    f.write("2026-07-08 09:01:25 LOGIN FAILED user=admin ip=203.0.113.99\n")
    f.write("2026-07-08 09:01:30 LOGIN FAILED user=admin ip=203.0.113.99\n")
    f.write("2026-07-08 09:01:35 LOGIN FAILED user=admin ip=203.0.113.99\n")
    f.write("2026-07-08 09:02:00 LOGIN SUCCESS user=bob ip=10.0.0.8\n")

def simple_ids(logfile, threshold=3):
    attempts = defaultdict(int)
    alerts = []
    with open(logfile) as f:
        for line in f:
            if "LOGIN FAILED" in line:
                ip = line.split("ip=")[1].strip()
                attempts[ip] += 1
                if attempts[ip] == threshold:
                    alerts.append(f"IDS ALERT: {threshold}+ failed logins from {ip}")
    return alerts

for alert in simple_ids("login_attempts.log", threshold=3):
    print(alert)
```

Sample Output:

```
IDS ALERT: 3+ failed logins from 203.0.113.99
```

Where to Run:

Any free compiler. The script creates login_attempts.log itself, so it runs standalone with no upload required.

Experiment 11: Secure Communication with Symmetric Encryption

Aim:

To simulate secure (VPN-like) communication between two parties using symmetric key encryption.

Concept:

VPNs encrypt traffic so eavesdroppers can't read it, even if they intercept it. This experiment uses the Fernet symmetric encryption scheme (from the 'cryptography' package) to encrypt a message with a shared key and decrypt it back.

Python Code:

```
from cryptography.fernet import Fernet
# Install once: pip install cryptography

# Step 1: Generate a shared secret key (both sender & receiver need this)
key = Fernet.generate_key()
cipher = Fernet(key)

# Step 2: Sender encrypts the message before sending over the network
message = b"Transfer $500 to account 12345"
encrypted = cipher.encrypt(message)
print("Encrypted (what an eavesdropper sees):", encrypted)

# Step 3: Receiver decrypts it using the same key
decrypted = cipher.decrypt(encrypted)
print("Decrypted (what the receiver reads):", decrypted.decode())
```

Sample Output:

```
Encrypted (what an eavesdropper sees): b'gAAAAABqTbmc...I0oIy6HYTXys3T80IY9giM8='
Decrypted (what the receiver reads): Transfer $500 to account 12345
```

Where to Run:

Replit or Google Colab only. Requires 'pip install cryptography' — not preinstalled on Programiz/OnlineGDB, which don't support package installs.

Experiment 12: Automated Incident Report Generator

Aim:

To automatically generate a simple, readable incident report from detected security events.

Concept:

After detecting an incident, responders document what happened for the record and for management. This experiment takes the flagged IPs from Experiment 7/10 and formats them into a basic text report — the final step of the incident response lifecycle.

Python Code:

```
import datetime

def generate_report(flagged_ips, source_log):
    lines = []
    lines.append("INCIDENT RESPONSE REPORT")
    lines.append(f"Generated: {datetime.datetime.now()}")
    lines.append(f"Source log: {source_log}")
    lines.append("-" * 40)
    if flagged_ips:
        lines.append(f"{len(flagged_ips)} suspicious IP(s) detected:")
        for ip, count in flagged_ips.items():
            lines.append(f"  - {ip}: {count} failed login attempts")
        lines.append("Recommended action: Block listed IPs, force password reset.")
    else:
        lines.append("No suspicious activity detected.")
    return "\n".join(lines)

flagged = {"203.0.113.99": 5}
report = generate_report(flagged, "login_attempts.log")
print(report)

with open("incident_report.txt", "w") as f:
    f.write(report)
```

Sample Output:

```
INCIDENT RESPONSE REPORT
Generated: 2026-07-08 08:14:44.360323
Source log: login_attempts.log
-----
1 suspicious IP(s) detected:
  - 203.0.113.99: 5 failed login attempts
Recommended action: Block listed IPs, force password reset.
```

Where to Run:

Any free compiler. Pure logic; writes its own report file.

Experiment 13: Basic Digital Forensics – File Tamper Detection

Aim:

To detect whether a file has been modified by comparing its hash against a known-good baseline.

Concept:

A key forensic technique is verifying evidence integrity using hashes. This experiment creates an original file, an identical copy, and a modified copy, then uses SHA-256 hashing to correctly identify which one was tampered with.

Python Code:

```
import hashlib

def hash_file(path):
    with open(path, "rb") as f:
        return hashlib.sha256(f.read()).hexdigest()

# Create sample files: one original, one copy, one tampered
with open("original.txt", "w") as f: f.write("System32 backup data")
with open("copy.txt", "w") as f: f.write("System32 backup data")
with open("tampered.txt", "w") as f: f.write("System32 backup data - modified")

files = ["original.txt", "copy.txt", "tampered.txt"]
hashes = {f: hash_file(f) for f in files}

baseline = hashes["original.txt"]
print("Baseline (original.txt):", baseline)
for f in files[1:]:
    status = "IDENTICAL" if hashes[f] == baseline else "MODIFIED/TAMPERED"
    print(f"{f}: {status}")
```

Sample Output:

```
Baseline (original.txt):
9ac62dc309d11594f1da0d160ded90eca26724461b1c55abeed293c23a0285d1
copy.txt: IDENTICAL
tampered.txt: MODIFIED/TAMPERED
```

Where to Run:

Any free compiler. The script creates its own sample files, so it runs standalone.

Appendix: Free Online Tools for Each Experiment

The table below summarizes which free online Python tool to use for each experiment. Recommended tools: Programiz Online Compiler, OnlineGDB, Trinket, Replit (replit.com), and Google Colab (colab.research.google.com). Replit and Google Colab support network access and pip installs, so they can run all 13 experiments without modification. Programiz, OnlineGDB, and Trinket work for experiments that need only the standard library and no network access.

Exp.	Title	Free Tool	Special Requirement
1	Caesar Cipher	Any (Programiz / OnlineGDB / Replit / Colab)	None
2	File Hashing	Any	None (self-contained)
3	TCP Port Scanner	Replit / Google Colab	Network access
4	Phishing URL Detector	Any	None
5	DNS Lookup (OSINT)	Replit / Google Colab	Network access
6	IoC Matching	Any	None (self-contained)
7	Log File Analyzer	Any	None (self-contained)
8	Dictionary Attack Demo	Any	None
9	Firewall Rule Simulator	Any	None
10	Signature-Based IDS	Any	None (self-contained)
11	Secure Communication (Fernet)	Replit / Google Colab	pip install cryptography
12	Incident Report Generator	Any	None
13	File Tamper Detection	Any	None (self-contained)

Tip: if you must use a bare compiler (Programiz/OnlineGDB) for Experiments 3, 5, or 11, either switch to Replit/Colab for just those three, or run them locally on your own machine with Python installed.